

Отчет по РБПО

- Студент: Оганесян Микаэл
- Группа: БПИ234

1) NFR (Требования безопасности) – docs/security-nfr

Проект демонстрирует полное и качественное выполнение всех требований по данному блоку.

а) Создано 8 нефункциональных требований:

- **NFR-01: Скорость загрузки (p95):** 95% запросов на получение главной страницы каталога. $p95 \leq 1000$ ms при 50 RPS.
- **NFR-02: Хеширование паролей:** Использование стойкого алгоритма для всех паролей пользователей (argon2-cffi).
- **NFR-03: Защита от Brute-Force:** Ограничение числа неудачных попыток входа (5 неудачных попыток за 1 минуту, блокировка на 10 минут).
- **NFR-04: Безопасное соединение:** 100% трафика через HTTPS.
- **NFR-05: Срок жизни токена:** Access Token TTL = 30 минут.
- **NFR-06: Уязвимости зависимостей:** Critical/High ≤ 14 дней.
- **NFR-07: Сохранение логов:** Хранение логов 30 дней.
- **NFR-08: Отказоустойчивость API:** Uptime $\geq 99.9\%$.

б) Созданы BDD-сценарии для проверки NFR (NFR_BDD):

- В docs/security-nfr/NFR_BDD.md представлено 4 BDD-сценария (NFR-01, NFR-03, NFR-02, NFR-08), каждый из которых включает секции Given-When-Then с конкретными критериями приемки и метриками.
 - **Примеры сценариев:**
 - ☐ NFR-01: Время загрузки каталога не превышает 1 секунду.
 - ☐ NFR-03: Блокировка IP после превышения лимита неудачных попыток входа.
 - ☐ NFR-02: Хеширование пароля при регистрации с использованием Argon2id.
 - ☐ NFR-08: Доступность критического API авторизации (Uptime 99.9%).

в) Матрица трассировки (NFR_TRACEABILITY):

- В docs/security-nfr/NFR_TRACEABILITY.md представлена матрица трассировки, связывающая каждое NFR с соответствующей Story/Task, приоритетом и сроком.
- **Стратегии проверки:** Указаны в docs/security-nfr/NFR.md.

2) Threat Modeling (Моделирование угроз) – docs/threat-model

Проект демонстрирует качественное моделирование угроз.

а) Диаграмма потоков данных (DFD):

- В docs/threat-model/DFD.md представлена диаграмма с 3 зонами (Границы доверия): Фронтенд, Бэкенд, База данных.
- Описано 6 основных потоков данных (F1-F6), таких как "Запрос страницы каталога", "Запрос данных каталога", "Запрос данных из БД" и другие, с детальным описанием каждого потока.
- Определены внешние сущности: Пользователь.

б) Анализ угроз (STRIDE):

- В docs/threat-model/STRIDE.md проведен детальный анализ угроз по методологии STRIDE, выявлено 9 угроз для различных потоков данных и компонентов.
 - **S: Spoofing** (Подмена): Для F1, F2, F5 (контроль: Безопасное соединение, NFR-04).
 - **T: Tampering** (Искажение): Для F1, F2, F5 (контроль: Безопасное соединение, NFR-04) и P3, P4 (Защита от уязвимостей в зависимостях, NFR-06).

- **R: Repudiation** (Отказ от авторства): Для F1, F2, F5 (контроль: Логирование критичных действий, NFR-07).
- **I: Information Disclosure** (Разглашение информации): Для F3, F4 (Защита от SQL-инъекций, NFR-04), P2, P3 (Контролируемый доступ к логам, NFR-07) и P3 (Защита от уязвимостей в зависимостях, NFR-06).
- **D: Denial of Service** (Отказ в обслуживании): Для P2, P3 (Мониторинг скорости загрузки, NFR-01, NFR-08).
- **E: Elevation of Privilege** (Повышение привилегий): Для P2, P3 (Хеширование паролей, защита от Brute-Force, NFR-02, NFR-03).
- Для каждой угрозы указан контроль, ссылка на NFR и обоснование/артефакт проверки.

с) Риски (RISKS):

- В `docs/threat-model/RISKS.md` составлен реестр из 10 ключевых рисков с количественной оценкой L (likelihood) и I (impact).
- Расчет Risk Score = $L \times I$, например, R-03 "SQL-инъекции" имеет Risk Score 12 ($L=3, I=4$).
- Для каждого риска определена стратегия управления (Снизить/Акцепт/Transfer/Avoid), например, для R-03 стратегия "Снизить".

3) ADR (Architecture Decision Records)

Проект использует ADR для документирования ключевых архитектурных решений, связанных с безопасностью.

а) ADR-001: Защита загрузки файлов и валидация ввода (`docs/adr/ADR-001-Input-Validation.md`)

- **Context:** Снижение рисков, связанных с приемом внешних данных (R-07, Path Traversal, вредоносные файлы).
- **Decision:** Внедрение централизованного модуля безопасной загрузки/валидации: проверка содержимого по сигнатурам, лимиты размера (`MAX_BYTES = 5MB`), использование UUID для имен файлов, канонизация путей.
- **Alternatives:** Ручная валидация в обработчиках FastAPI, использование обособленной библиотеки валидации.
- **Links:** NFR-01, R-07, R-09.
- **Tests:** `tests/test_security.py::test_rejects_big_file`, `tests/test_security.py::test_sniffs_bad_type`. В `tests/test_errors.py` есть тест `test_rejects_invalid_input_422_rfc7807`, который покрывает часть валидации.

б) ADR-002: Стандартизация ошибок по RFC 7807 (`docs/adr/ADR-002-RFC7807-Errors.md`)

- **Context:** Повышение предсказуемости API и предотвращение раскрытия чувствительных деталей внутренней реализации (угроза Information Disclosure, риск R-04).
- **Decision:** Все HTTP-ответы об ошибках (4xx, 5xx) в формате RFC 7807 (Problem Details for HTTP APIs). JSON с `Content-Type: application/problem+json`, обязательные поля (`type`, `title`, `status`, `detail`), `correlation_id` (UUIDv4) для трассировки, маскирование `detail` от трассировки стека.
- **Alternatives:** Кастомный формат JSON-ответа, использование стандартных исключений FastAPI без перехвата.
- **Implementation:** `app/main.py` содержит `@app.exception_handler` для `Exception`, `HTTPException` и `RequestValidationError`, которые используют функцию `problem` из `app/utils/errors.py` для формирования ответов в формате RFC 7807.
- **Tests:** `tests/test_errors.py` содержит тесты `test_not_found_item_rfc7807` и `test_rejects_invalid_input_422_rfc7807`, которые проверяют соответствие формату RFC 7807.
- **Links:** NFR-07, R-04.

с) ADR-003: Управление секретами: Переменные окружения (`docs/adr/ADR-003-Handling-Secrets.md`)

- **Context:** Устранение риска R-06 (Несанкционированный прямой доступ к БД) и обеспечение безопасности критических данных.
- **Decision:** Все секреты загружаются исключительно из переменных окружения. Запрет на хранение секретов в Git или жесткое кодирование. Сервис должен падать (`fail-fast`), если необходимая переменная окружения отсутствует (например, `DB_PASSWORD`).
- **Alternatives:** Использование файлов `.env`, использование Секрет-Менеджеров.

- **Implementation:** `app/db/database.py` использует `os.environ.get("DATABASE_URL")` и явно вызывает `OSError`, если переменная не установлена, что соответствует принципу `fail-fast`.
- **Tests:** `tests/test_errors.py` содержит тест `test_db_secret_env_only`, который проверяет, что приложение корректно падает при отсутствии переменной окружения `DATABASE_URL`.
- **Links:** NFR-07, R-06.

4) Приоритизация/оформление

- Во всех вышеуказанных файлах (`docs/security-nfr/NFR.md`, `docs/threat-model/DFD.md`, `docs/threat-model/STRIDE.md`, `docs/threat-model/RISKS.md`, `docs/security-nfr/NFR_BDD.md`, `docs/security-nfr/NFR_TRACEABILITY.md`, `docs/adr/ADR-001-Input-Validation.md`, `docs/adr/ADR-002-RFC7807-Errors.md`, `docs/adr/ADR-003-Handling-Secrets.md`) информация структурирована и оформлена согласно лучшим практикам.
 - Использование таблиц, четких заголовков и ссылок обеспечивает высокую читаемость и возможность трассировки.
 - **Git:** Проект находится в ветке `p05-secure-coding`, что указывает на организованную работу с версиями. Разработка велась в отдельных `practise`-ветках, которые затем вливались в `main` после ревью.
-

Заключение

Проект демонстрирует глубокое понимание и уверенное применение принципов безопасной разработки и DevSecOps. Особого внимания заслуживают:

- **Комплексное моделирование угроз:** Детальные NFR, DFD, анализ STRIDE и реестр рисков с трассировкой показывают системный подход к безопасности.
- **Документирование архитектурных решений (ADR):** ADRs по валидации ввода, стандартизации ошибок и управлению секретами отражают зрелый подход к принятию и документированию важных решений.
- **Реализация безопасного кодирования:** Использование Pydantic для валидации и SQLAlchemy ORM для защиты от SQLi является сильным показателем.
- **Укрепление контейнеров:** Multi-stage сборка Dockerfile, использование `minimal`-образа и `non-root` пользователя повышают безопасность приложения в контейнере.